ASSIGNMENT - 03 - CO3

(Lists, Tuples, Dictionaries, Sets and Nested Data)

COURSE NAME : Python Programming

COURSE CODE : CSA0825

COURSE FACULTY : Hariramachandran T

SLOT:A

SUBMITTED BY : G.anushka(1912260668)

GITHUB : https://github.com/anushkapearl21-jpg/CSA0825-CO3.git

Q1 – Lists and Slicing: Daily Rainfall Records 6 Marks

Bloom's Level: K3 – Apply

Aim:

To store 365 rainfall readings, calculate weekly totals, identify the wettest and driest weeks, calculate 7-day moving averages and list dry days.

Algorithm:

1. Generate or read the daily rainfall values and store them in a list.
2. Use slicing in groups of seven to calculate weekly totals.
3. Use enumerate() with max() and min() to identify the wettest and driest weeks.
4. Calculate moving averages using both the naive method and a running sum.
5. Find the day numbers for which the reading is zero.
6. Test empty and shorter-than-seven input lists and compare execution times.

Code:

```
import random
import time

def weekly_totals(readings):
    return [sum(readings[i:i+7]) for i in range(0,
len(readings), 7)]

def moving_naive(values, window=7):
    if len(values) < window:
        return []
    return [sum(values[i:i+window]) / window
            for i in range(len(values)-window+1)]

def moving_running(values, window=7):
    if len(values) < window:
        return []
    result = []
    current = sum(values[:window])
    result.append(current / window)
    for i in range(window, len(values)):
        current += values[i] - values[i-window]
        result.append(current / window)
    return result

random.seed(10)
readings = [random.randint(0, 20) for _ in range(365)]
```

```
weeks = weekly_totals(readings)
wettest = max(enumerate(weeks, start=1), key=lambda x: x[1])
driest = min(enumerate(weeks, start=1), key=lambda x: x[1])
averages = moving_running(readings)
dry_days = [day for day, value in enumerate(readings,
start=1) if value == 0]

print("Number of readings:", len(readings))
print("Number of weekly totals:", len(weeks))
print("First 3 weekly totals:", weeks[:3])
print("Wettest week:", wettest)
print("Driest week:", driest)
print("First 3 moving averages:", [round(x, 2) for x in
averages[:3]])
print("Number of dry days:", len(dry_days))
print("Empty input:", moving_running([]))
print("Short input:", moving_running([2, 3, 4]))

for n in (365, 36500, 365000):
    data = [random.randint(0, 20) for _ in range(n)]
    start = time.perf_counter()
    a = moving_naive(data)
    naive_time = time.perf_counter() - start
    start = time.perf_counter()
    b = moving_running(data)
    running_time = time.perf_counter() - start
    print(n, "equal:", a == b,
          "naive:", round(naive_time, 4),
          "running:", round(running_time, 4))
```

Output:

```
Number of readings: 365
Number of weekly totals: 53
First 3 weekly totals: [71, 79, 59]
Wettest week: (8, 99)
Driest week: (53, 9)
First 3 moving averages: [10.14, 9.57, 11.57]
Number of dry days: 9
Empty input: []
Short input: []
365 equal: True naive: 0.0001 running: 0.0
36500 equal: True naive: 0.0061 running: 0.0024
365000 equal: True naive: 0.0523 running: 0.0221
```

Design, Analysis and Observation:

Pseudocode for the moving average:

If fewer than 7 readings are available, return an empty list. Otherwise, add the first seven readings and store their average. For every next day, subtract the reading that leaves the window and add the new reading. Store the updated sum divided by 7.

Meaning of the slices:

readings[7:14] returns elements at index 7 through 13 (the eighth to fourteenth readings). readings[-7:] returns the final seven readings. readings[::7] returns every seventh element, starting at index 0; it does not calculate weekly totals.

Why a list is suitable:

A list keeps the 365 daily readings in order and supports indexing and slicing. append() is O(1) amortised, insert() is O(n) because later elements may need shifting, pop() at the end is O(1) while pop(i) is O(n), and sort() is O(n log n).

Addition count and observation:

For n readings and a window of 7, the naive method performs about 7(n−6) additions. The running-sum method performs 7 additions initially and then one addition plus one subtraction for each of the remaining n−7 readings. Both methods return the same averages, but the running sum scales linearly with a smaller constant amount of work.

Late reading decision:

list.insert() at the correct date costs O(n) in the worst case because elements after the insertion position are shifted. A list is still suitable for a small year-long record. For frequent late arrivals in a large stream, a date-keyed dictionary or a database table may be more appropriate.

Alternative Concept and Complexity Analysis:

A collections.deque(maxlen=7) can maintain the latest seven readings as a rolling window. Adding and removing at its ends is O(1), and the average can be updated with a running sum in O(1) per new reading. A dictionary keyed by date makes late-reading updates easy to locate, but weekly reports require sorting or grouping dates. For n days, a naive moving average is O(7n), which is O(n) for a fixed window; a running sum is O(n). A list uses O(n) space, while a streaming deque uses O(7) space if old readings need not be retained.

Q2 – Tuples, Mutability and Copying 7 Marks

Bloom's Level: K4 – Analyse

Aim:

To represent bus-stop coordinates using tuples, correct a shared-row grid bug, demonstrate aliasing and copying, and compare tuples with lists.

Algorithm:

1. Represent each bus stop as a latitude-longitude tuple and use it as a dictionary key.
2. Create a grid using a list comprehension so that each row is independent.
3. Demonstrate the repeated-reference bug using list multiplication.
4. Compare aliasing, shallow copy and deep copy.
5. Unpack a tuple in a for loop and attempt to use a list as a dictionary key.
6. Use sys.getsizeof() to compare large list and tuple containers.

Code:

```
import copy
import sys
import time

rows, cols = 3, 4
bad = [[0] * cols] * rows
bad[0][1] = 7
good = [[0 for _ in range(cols)] for _ in range(rows)]
good[0][1] = 7

print("bad :", bad)
print("good:", good)
print("same row in bad:", bad[0] is bad[1])
print("same row in good:", good[0] is good[1])

stops = {
    (13.0827, 80.2707): "Chennai Central",
    (13.0418, 80.2341): "T. Nagar"
}
print("Stop:", stops[(13.0418, 80.2341)])

try:
    stops[[13.0418, 80.2341]] = "Invalid key"
except TypeError as error:
    print(type(error).__name__ + ":", error)

original = [[1, 2], [3, 4]]
shallow = copy.copy(original)
deep = copy.deepcopy(original)
```

```
original[0][0] = 99
print("shallow shares inner list:", shallow[0] is
original[0])
print("deep shares inner list:", deep[0] is original[0])
print("shallow:", shallow)
print("deep:", deep)

packed = (10, 20)
x, y = packed
print("unpacked:", x, y)

large_list = list(range(1000000))
large_tuple = tuple(range(1000000))
print("list bytes:", sys.getsizeof(large_list))
print("tuple bytes:", sys.getsizeof(large_tuple))
```

Output:

```
bad : [[0, 7, 0, 0], [0, 7, 0, 0], [0, 7, 0, 0]]
good: [[0, 7, 0, 0], [0, 0, 0, 0], [0, 0, 0, 0]]
same row in bad: True
same row in good: False
Stop: T. Nagar
TypeError: unhashable type: 'list'
shallow shares inner list: True
deep shares inner list: False
shallow: [[99, 2], [3, 4]]
deep: [[1, 2], [3, 4]]
unpacked: 10 20
list bytes: 8000056
tuple bytes: 8000040
```

Design, Analysis and Observation:

Why tuples can be dictionary keys:

A tuple can be used as a dictionary key when all its elements are hashable. Its contents cannot be reassigned, so its hash remains stable. A list is mutable and unhashable, so Python raises TypeError when it is used as a key.

Memory diagram and aliasing:

In [[0] * 3] * 3, the outer list contains three references to the same inner list. Updating one cell therefore changes all rows. The corrected list comprehension creates a new inner list for every row. The is operator checks object identity, while id() displays an object's identity value during that run.

Shallow copy and deep copy:

A shallow copy creates a new outer container but keeps references to the original nested objects. It is enough when the elements are immutable or when shared nested objects are intended. A deep copy recursively copies nested objects and is needed when independent nested data is required.

Function arguments and mutable values:

A function receives a reference to the same list object. If the function appends or changes an element, the caller sees that change. Reassigning the local parameter to a new list does not replace the caller's variable.

Memory and timing investigation:

sys.getsizeof() reports the size of the container itself, not the complete memory occupied by all referenced integer objects. Exact byte counts vary by Python implementation and platform. Tuple and list creation/indexing timings can be measured with time.perf_counter(); both support O(1) indexing, while a tuple is generally smaller because it does not need list over-allocation for future appends.

Alternative Concept and Complexity Analysis:

A sparse zone grid can be represented by a dictionary keyed by (row, column) tuples. Only zones with passengers need entries, so the space becomes O(k), where k is the number of non-empty zones, instead of O(rows × columns). Average dictionary access is O(1), similar to indexing a list grid. A normal list grid is simpler when most zones are used; the dictionary is better when the grid is large and mostly empty.

Q3 – Dictionaries: Farmer Helpline Complaint Analysis 7 Marks

Bloom's Level: K4 – Analyse

Aim:

To analyse farmer complaints by counting frequent words, grouping complaints by district and finding the top words while ignoring common words.

Algorithm:

1. Convert each complaint to lowercase and extract words without punctuation.
2. Skip words that are present in the stop-word set.
3. Use dict.get() to increment each word count.
4. Use setdefault() to group complaints by district.
5. Sort the word-count pairs by descending count and alphabetical order for ties.
6. Compare the dictionary result with collections.Counter at different input sizes.

Code:

```
import re
import random
import time
from collections import Counter

calls = [
    "seeds not received seeds delayed",
    "fertiliser price high",
    "seeds price high subsidy not received",
]
STOP = {"not", "high"}

def frequencies(texts, stop):
    counts = {}
    for text in texts:
        words = re.findall(r"[a-z]+", text.lower())
        for word in words:
            if word not in stop:
                counts[word] = counts.get(word, 0) + 1
    return counts

def top(counts, k):
    return sorted(counts.items(), key=lambda item: (-
item[1], item[0]))[:k]

def group_by_district(records):
    grouped = {}
    for district, complaint in records:
        grouped.setdefault(district, []).append(complaint)
```

```
    return grouped

print(top(frequencies(calls, STOP), 3))
records = [
    ("Thiruvallur", "seeds delayed"),
    ("Chennai", "water supply"),
    ("Thiruvallur", "fertiliser price")
]
print(group_by_district(records))

# Scale test: generated complaint sentences
random.seed(4)
vocabulary = ["seed", "crop", "water", "price", "loan",
"pest", "soil", "yield"]
for n in (1000, 10000, 100000):
    texts = [" ".join(random.choices(vocabulary, k=8)) for _
in range(n)]
    start = time.perf_counter()
    result = frequencies(texts, set())
    dict_time = time.perf_counter() - start
    start = time.perf_counter()
    counter_result = Counter(word for text in texts for word
in text.split())
    counter_time = time.perf_counter() - start
    print(n, "dictionary words:", sum(result.values()),
          "Counter words:", sum(counter_result.values()),
          "dictionary:", round(dict_time, 4),
          "Counter:", round(counter_time, 4))
```

Output:

```
[('seeds', 3), ('price', 2), ('received', 2)]
{'Thiruvallur': ['seeds delayed', 'fertiliser price'], 'Chennai': ['water supply']}
1000 dictionary words: 8000 Counter words: 8000 dictionary: 0.0017 Counter: 0.0009
10000 dictionary words: 80000 Counter words: 80000 dictionary: 0.015 Counter: 0.007
100000 dictionary words: 800000 Counter words: 800000 dictionary: 0.1389 Counter: 0.06
66
```

Design, Analysis and Observation:

Dictionary lookup:

Python dictionaries use a hash table. The key's hash helps select a location, and equality checks resolve collisions. Lookup, insertion and update take O(1) average time, so the average does not grow linearly with the number of stored keys. Worst-case behaviour can be slower when many collisions occur.

Dictionary versus list.count():

A dictionary updates a word's count during one pass. If there are n total words and d distinct words, repeated list.count() for every distinct word takes O(n × d), while dictionary counting takes O(n) average time. The dictionary uses O(d) space.

Sorting and key views:

The top() function sorts by negative count and then alphabetically. Therefore, price comes before received because both have count 2 and 'price' comes earlier alphabetically. In Python 3.7 and later, dictionaries preserve insertion order. keys(), values() and items() return dynamic view objects that reflect later dictionary changes.

Engineering decision and tests:

For 100,000 complaints per day, I would deploy dictionary counting or collections.Counter. Both perform a single pass and are much more suitable than repeatedly scanning a list. Converting text to lowercase and extracting alphabetic words handles upper/lower case and punctuation consistently. Empty text produces an empty dictionary.

Alternative Concept and Complexity Analysis:

collections.Counter is a specialised dictionary subclass designed for counting. It provides concise operations such as most_common(), but explicit sorting is still useful when alphabetical tie-breaking is required. For n words and d distinct words, a dictionary loop and Counter both take O(n) average time and O(d) space. list.count() for each distinct word takes O(n × d) time and O(d) space for the resulting counts.

Q4 – Sets: Elective Registration Clash Detection 7 Marks

Bloom's Level: K4 – Analyse

Aim:

To detect elective registration clashes using set operations, remove duplicate registrations and produce sorted reports for students in both, exactly one, or neither elective.

Algorithm:

1. Store the register numbers for each elective in a set.
2. Create the universal set of all students.
3. Use intersection to find students in both electives.
4. Use symmetric difference to find students in exactly one elective.
5. Use difference after union to find students in neither elective.
6. Repeat the operations with set methods and compare the results.
7. Measure membership checks in a list and a set.

Code:

```
ai_elective = {
    "192311001", "192311004", "192311007", "192311009"
}
iot_elective = {
    "192311002", "192311004", "192311009", "192311010"
}
all_students = {f"1923110{i:02d}" for i in range(1, 11)}

both = ai_elective & iot_elective
exactly_one = ai_elective ^ iot_elective
neither = all_students - (ai_elective | iot_elective)

print("clash      :", sorted(both))
print("exactly one:", sorted(exactly_one))
print("neither    :", sorted(neither))

# Repeat with set methods
both_method = ai_elective.intersection(iot_elective)
one_method = ai_elective.symmetric_difference(iot_elective)
neither_method =
all_students.difference(ai_elective.union(iot_elective))
print("same results:", both == both_method and
      exactly_one == one_method and neither ==
neither_method)

# Membership comparison
import time
```

```
numbers = [str(i) for i in range(100000)]
number_set = set(numbers)
target = "99999"
start = time.perf_counter()
target in numbers
list_time = time.perf_counter() - start
start = time.perf_counter()
target in number_set
set_time = time.perf_counter() - start
print("membership checked; list time:", round(list_time, 6))
print("membership checked; set time:", round(set_time, 6))
```

Output:

```
clash      : ['192311004', '192311009']
exactly one: ['192311001', '192311002', '192311007', '192311010']
neither    : ['192311003', '192311005', '192311006', '192311008']
same results: True
membership checked; list time: 0.000792
membership checked; set time: 1e-06
```

Design, Analysis and Observation:

Set operations:

The intersection (&) gives students in both electives. The symmetric difference (^) gives students in exactly one elective. The union (|) combines all registered students, and subtracting that union from all_students gives students in neither elective. sorted() is used to produce a readable report.

Duplicates and membership:

A set stores each distinct hashable element only once, so duplicate register numbers are removed automatically. Membership testing in a set is O(1) average because it uses hashing; membership in a list is O(n) because values may be checked one by one.

Venn diagram interpretation:

The overlap of the two circles represents the intersection. The non-overlapping parts of both circles together represent the symmetric difference. The area outside both circles but inside the universal set represents students who registered for neither elective.

Limitations:

A set does not preserve duplicate counts or the original order of registrations. Sets cannot contain lists because lists are mutable and unhashable, but they can contain tuples when every tuple element is hashable.

Alternative Concept and Complexity Analysis:

If electives are stored as lists, nested loops require O(n × m) comparisons to find common students. Sorting both lists and walking through them takes O(n log n + m log m) time. Converting the lists to sets gives O(n + m) average time for construction and set operations, with O(n + m) additional space. Sets are the preferred choice when duplicate registrations and original order are not important.

Q5 – List Comprehension: Cleaning Sensor Readings 6 Marks

Bloom's Level: K3 – Apply

Aim:

To remove invalid cold-storage temperature readings, convert valid Celsius readings to Fahrenheit, and identify minutes outside the safe 2–8 ℃ range.

Algorithm:

1. Read the temperature values into a list.
2. Keep only values from −50 ℃ through 50 ℃ as valid sensor readings.
3. Convert each valid Celsius value to Fahrenheit using F = C × 9/5 + 32.
4. Implement the operation using a loop, list comprehension, and map() with filter().
5. Use enumerate() to identify valid readings outside the safe 2–8 ℃ range.
6. Compare the outputs and execution times for 10,000, 100,000 and 1,000,000 readings.

Code:

```
import random
import time
import sys

def clean_loop(readings):
    celsius = []
    fahrenheit = []
    for value in readings:
        if -50 <= value <= 50:
            celsius.append(value)
            fahrenheit.append(value * 9 / 5 + 32)
    return celsius, fahrenheit

def clean_comprehension(readings):
    celsius = [v for v in readings if -50 <= v <= 50]
    fahrenheit = [v * 9 / 5 + 32 for v in celsius]
    return celsius, fahrenheit

def clean_map_filter(readings):
    valid = list(filter(lambda v: -50 <= v <= 50, readings))
    fahrenheit = list(map(lambda v: v * 9 / 5 + 32, valid))
    return valid, fahrenheit

readings = [4, 5, 55, -60, 7, 9, 1, 51, -2]
c1, f1 = clean_loop(readings)
c2, f2 = clean_comprehension(readings)
c3, f3 = clean_map_filter(readings)

alarm_minutes = [
```

```
    minute for minute, temp in enumerate(readings, start=1)
    if -50 <= temp <= 50 and not (2 <= temp <= 8)
]
fahrenheit_map = {i: c * 9 / 5 + 32
                  for i, c in enumerate(c1, start=1)}
valid_set = {v for v in readings if -50 <= v <= 50}

print("valid Celsius:", c1)
print("valid Fahrenheit:", [round(x, 1) for x in f1])
print("alarm minutes:", alarm_minutes)
print("all methods equal:", c1 == c2 == c3 and f1 == f2 ==
f3)
print("dictionary entries:", len(fahrenheit_map))
print("unique valid readings:", sorted(valid_set))

for n in (10000, 100000, 1000000):
    data = [random.uniform(-60, 60) for _ in range(n)]
    times = []
    results = []
    for function in (clean_loop, clean_comprehension,
clean_map_filter):
        start = time.perf_counter()
        result = function(data)
        times.append(time.perf_counter() - start)
        results.append(result)
    print(n, "equal:", results[0] == results[1] ==
results[2],
          "times:", [round(t, 4) for t in times])

gen = (x for x in readings if -50 <= x <= 50)
print("generator object bytes:", sys.getsizeof(gen))
print("list object bytes:", sys.getsizeof(c1))
```

Output:

```
valid Celsius: [4, 5, 7, 9, 1, -2]
valid Fahrenheit: [39.2, 41.0, 44.6, 48.2, 33.8, 28.4]
alarm minutes: [6, 7, 9]
all methods equal: True
dictionary entries: 6
unique valid readings: [-2, 1, 4, 5, 7, 9]
10000 equal: True times: [0.0013, 0.0012, 0.0016]
100000 equal: True times: [0.0105, 0.0091, 0.0117]
1000000 equal: True times: [0.0931, 0.0947, 0.1205]
generator object bytes: 208
list object bytes: 120
```

Design, Analysis and Observation:

List comprehension form:

[expression for item in iterable if condition] creates a new list containing the expression result for every item that satisfies the condition. A comprehension is concise for simple filtering and transformation. A normal loop is easier to read when several steps, error handling or nested conditions are required.

Alarm calculation:

enumerate(readings, start=1) attaches a one-based minute number to each reading. A minute is an alarm minute when the reading is valid but outside the safe 2–8 °C range. Sensor errors below −50 °C or above 50 °C are excluded from the valid temperature list.

Comprehension, map/filter and generator:

The loop, list-comprehension and map/filter versions produce the same valid Celsius and Fahrenheit values. A list comprehension stores all results immediately. A generator expression produces values lazily and normally uses O(1) additional iterator memory, but it can be consumed only once. sys.getsizeof() reports the generator object size, not the total memory of values it will later produce.

Conclusion:

A list comprehension is not always the best choice. It is clear for short transformations, but a loop may be easier to maintain for complex logic. A generator is useful when values can be processed one at a time and the complete result does not need to be stored.

Alternative Concept and Complexity Analysis:

A generator expression can be used when the results are consumed once, for example sum(c for c in readings if -50 <= c <= 50). Both list comprehension and generator processing take O(n) time. A list requires O(n) memory for the results, while a generator requires O(1) additional iterator memory (excluding the source data). The generator is preferable for streaming or aggregation; a list is preferable when results must be reused or indexed.

Q6 – Nested Data: A Railway Timetable 7 Marks

Bloom's Level: K4 – Analyse

Aim:

To build a nested railway timetable using dictionaries, lists, tuples and sets, find trains stopping at a selected station on a selected day, and sort them by departure time.

Algorithm:

1. Create a dictionary containing at least five trains.
2. For each train, store its name, ordered stop tuples and set of running days.
3. Scan the timetable to find trains stopping at the requested station on the requested day.
4. Sort matching trains using sorted() and key=lambda based on departure time.
5. Build a station-to-train index and repeat the query using the index.
6. Test an unknown station and a train that does not run on the selected day.

Code:

```
from collections import defaultdict
import time
import random

timetable = {
    "12601": {
        "name": "Chennai Mail",
        "stops": [
            ("Chennai", "06:00", "06:10"),
            ("Arakkonam", "07:20", "07:22"),
            ("Katpadi", "08:30", "08:32")
        ],
        "days": {"Mon", "Tue", "Wed", "Thu", "Fri", "Sat",
"Sun"}
    },
    "12602": {
        "name": "Coimbatore Express",
        "stops": [
            ("Chennai", "07:00", "07:15"),
            ("Salem", "11:00", "11:05"),
            ("Coimbatore", "13:00", "13:10")
        ],
        "days": {"Mon", "Wed", "Fri"}
    },
    "12603": {
        "name": "Bengaluru Intercity",
        "stops": [
            ("Chennai", "08:00", "08:05"),
            ("Katpadi", "10:00", "10:02"),
```

```
            ("Bengaluru", "13:00", "13:10")
        ],
        "days": {"Tue", "Thu", "Sat"}
    },
    "12604": {
        "name": "Madurai Express",
        "stops": [
            ("Chennai", "20:00", "20:15"),
            ("Villupuram", "22:00", "22:05"),
            ("Madurai", "05:00", "05:10")
        ],
        "days": {"Mon", "Tue", "Wed", "Thu", "Fri", "Sat",
"Sun"}
    },
    "12605": {
        "name": "Tirupati Passenger",
        "stops": [
            ("Chennai", "09:00", "09:10"),
            ("Arakkonam", "10:30", "10:32"),
            ("Tirupati", "12:00", "12:10")
        ],
        "days": {"Mon", "Tue", "Wed", "Thu", "Fri"}
    }
}

def build_index(data):
    index = defaultdict(list)
    for number, info in data.items():
        for station, arrival, departure in info["stops"]:
            index[station].append(number)
    return index

def find_trains(data, station, day):
    matches = []
    for number, info in data.items():
        if day in info["days"]:
            for stop, arrival, departure in info["stops"]:
                if stop == station:
                    matches.append((departure, number,
info["name"]))
    return sorted(matches, key=lambda item: item[0])

index = build_index(timetable)
station, day = "Katpadi", "Tue"
print("Trains at", station, "on", day + ":")
for departure, number, name in find_trains(timetable,
station, day):
    print(number, name, "departure", departure)

print("Third stop departure of 12601:",
      timetable["12601"]["stops"][2][2])
print("Unknown station:", index.get("Unknown", []))
print("Index for Katpadi:", index["Katpadi"])
```

```
# Demonstrate indexed filtering
indexed_matches = [
    number for number in index.get(station, [])
    if day in timetable[number]["days"]
]
print("Indexed matches:", sorted(indexed_matches))
```

Output:

```
Trains at Katpadi on Tue:
12601 Chennai Mail departure 08:32
12603 Bengaluru Intercity departure 10:02
Third stop departure of 12601: 08:32
Unknown station: []
Index for Katpadi: ['12601', '12603']
Indexed matches: ['12601', '12603']
```

Design, Analysis and Observation:

Why each structure is used:

The outer dictionary provides fast access by train number. Each train's inner dictionary stores named fields such as name, stops and running days. A list preserves the order of stations. Each station entry is a tuple because station, arrival and departure form one fixed record. A set stores running days and provides fast membership checks.

Nested expression:

The expression timetable["12601"]["stops"][2][2] reaches the departure time of the third station for train 12601. The first key selects the train, "stops" selects its stop list, index 2 selects the third stop, and index 2 of that tuple selects departure time.

Query work and index:

Without an index, the program checks every train and each of its stops, giving O(S) work where S is the total number of timetable stops. A station index maps each station to the trains that stop there. A query then examines only the trains listed for that station, approximately O(k), where k is the number of matching trains, plus sorting.

Trade-off:

The station index uses additional memory proportional to the number of station-train stop relationships. It speeds up repeated station queries, but it must be updated whenever a train's stops change.

Alternative Concept and Complexity Analysis:

A flat list of (train, station, arrival, departure, days) tuples is simple and can be scanned directly, but every query examines all stop records, O(S). A nested dictionary organises data by train, while a station index makes a query proportional to the number of trains at that station, O(k), after the index is built. The index uses O(S) additional space. A flat list is suitable for small timetables; indexed nested data is better for repeated queries on a large timetable.

Q7 – Design a Contact-Merging Program from First Principles 10 Marks

Bloom's Level: K5 – Evaluate

Aim:

To design and evaluate a contact-merging program that normalises phone numbers and names, combines duplicate records, and stores unique phone numbers and e-mail addresses.

Algorithm:

1. Identify the input, output and matching rules for deciding whether two contacts are the same person.
2. Choose a dictionary keyed by a normalised phone number and sets for phone numbers and e-mail addresses.
3. Normalise phone numbers by removing spaces and punctuation and handling the +91 prefix.
4. Merge records by updating the sets stored for each dictionary key.
5. Test duplicate contacts, missing fields and different phone-number formats.
6. Measure the program with 1,000 and 100,000 contacts and compare it with pairwise comparison conceptually.

Code:

```
import re
import time

def normalise_phone(number):
    digits = re.sub(r"\D", "", number)
    if digits.startswith("91") and len(digits) == 12:
        digits = digits[2:]
    return digits


def normalise_name(name):
    return " ".join(name.lower().split())


def merge_contacts(contact_lists):
    merged = {}
    phone_to_key = {}

    for contact in contact_lists:
        numbers = {
            normalise_phone(n) for n in contact.get("phones", []) if n
        }
        emails = {
            e.strip().lower() for e in contact.get("emails", []) if e.strip()
        }

        # Find an existing group through ANY phone number in this record.
        existing = sorted({phone_to_key[n] for n in numbers if n in phone_to_key})
        if existing:
            key = existing[0]
```

```
            # Combine groups if this record connects two earlier groups.
            for old_key in existing[1:]:
                merged[key]["phones"].update(merged[old_key]["phones"])
                merged[key]["emails"].update(merged[old_key]["emails"])
                for old_phone in merged[old_key]["phones"]:
                    phone_to_key[old_phone] = key
                del merged[old_key]
        else:
            key = next(iter(sorted(numbers)), "name:" +
normalise_name(contact.get("name", "")))
            if key not in merged:
                merged[key] = {
                    "name": contact.get("name", "").strip(),
                    "phones": set(),
                    "emails": set()
                }

        merged[key]["phones"].update(numbers)
        merged[key]["emails"].update(emails)
        if not merged[key]["name"] and contact.get("name"):
            merged[key]["name"] = contact["name"].strip()
        for number in numbers:
            phone_to_key[number] = key

    return merged

contacts = [
    {"name": "Dr. Ravi", "phones": ["+91 98400 12345"],
     "emails": ["RAVI@example.com"]},
    {"name": "ravi kumar", "phones": ["9840012345", "9000011111"],
     "emails": ["ravi@example.com", "rk@example.com"]},
    {"name": "Meena", "phones": ["9888877777"], "emails": []},
    {"name": "Unknown", "phones": [], "emails": ["unknown@example.com"]}
]

merged = merge_contacts(contacts)
for key, value in merged.items():
    print(key, "=>", value["name"],
          sorted(value["phones"]), sorted(value["emails"]))

print("same phone normalises:",
      normalise_phone("+91 98400 12345") == normalise_phone("9840012345"))

# Scaling experiment with unique contacts
for n in (1000, 100000):
    data = [
        {"name": f"Person {i}", "phones": [f"90000{i:06d}"],
         "emails": [f"person{i}@example.com"]}
        for i in range(n)
    ]
    start = time.perf_counter()
    result = merge_contacts(data)
    elapsed = time.perf_counter() - start
    print(n, "contacts merged:", len(result),
          "time:", round(elapsed, 4), "seconds")
```

Output:

```
9840012345 => Dr. Ravi ['9000011111', '9840012345'] ['ravi@example.com', 'rk@example.c
om']
9888877777 => Meena ['9888877777'] []
name:unknown => Unknown [] ['unknown@example.com']
same phone normalises: True
1000 contacts merged: 1000 time: 0.0021 seconds
100000 contacts merged: 100000 time: 0.3395 seconds
```

Design, Analysis and Observation:

Stage 1 – Interpret the problem:

Input: contact records from two phones and a SIM card, containing names, phone numbers and e-mail addresses. Output: merged contact records with one entry for each identified person and all known phone numbers and e-mail addresses. The primary matching rule is a normalised phone number. Data structures: a dictionary for lookup and sets for unique phone numbers and e-mail addresses. Measurable requirements: equivalent phone formats must normalise to the same digits; repeated numbers and e-mails must appear once; missing fields must not crash the program.

Stage 2 – Design and pseudocode:

For every contact, remove spaces and punctuation from each phone number and remove the Indian country prefix 91 when it is present with a 12-digit number. Convert e-mail addresses to lowercase and trim spaces. Use a dictionary keyed by a normalised phone number. Add all numbers and e-mails to sets. If no phone exists, use a name-based fallback key. Pseudocode: FOR each contact, normalise its numbers and e-mails; choose a stable key; create a dictionary entry if absent; update the entry's phone and e-mail sets; RETURN the merged dictionary.

Stage 3 – Implementation and validation:

The test data contains duplicate contacts with different name spellings and phone formats, a contact with no e-mail, and a contact with no phone. The two Ravi records merge because their phone numbers normalise to 9840012345. A small result can be checked manually against the original records. The program also measures 1,000 and 100,000 contacts; exact timing depends on the computer.

Stage 4 – Defend the design:

A dictionary lookup takes O(1) average time, so processing n contacts is O(n) average, excluding the length of the text being normalised. Comparing every pair requires $O(n^2)$ comparisons. Two different people with the same name are not merged when their phone numbers differ, because the phone number is the primary key. A limitation is that two people sharing one family phone number may be merged incorrectly; a more advanced system should use multiple identifiers and manual confirmation.

Alternative Concept and Complexity Analysis:

Pairwise comparison checks each contact against the others and takes $O(n^2)$ time with O(n) storage for the result. A dictionary keyed by a normalised phone number takes O(n) average

time and O(n) space for n contacts, plus the storage for all unique numbers and e-mails. Dictionary grouping is therefore more suitable for large contact lists. Its correctness depends on the matching key: shared or recycled phone numbers can cause false merges, while contacts with no phone number need a cautious fallback or manual review.